TECHNICAL REPORT

mmcFoam v2306

Stochastic Particle Methods for Turbulent, Reacting, Multiphase Flows
– Performance and Usability –

Jan Wilhelm Gärtner

November 8, 2023

Contents

1 Introduction **1**
1.1 MMC Modeling . . . 1
1.2 Structure of the report . . . 2

2 OpenFOAM Changes from 5.x to v2306 **4**
2.1 Particle chemistry models . . . 4
2.1.1 ODE solver and thermo library . . . 5
2.2 Particle Tracking . . . 6
2.3 Other changes . . . 6

3 Performance &Usability Improvements **8**
3.1 Particle management and super mesh . . . 8
3.1.1 Possible improvements . . . 9
3.2 Particle pairing and mixing . . . 10
3.2.1 Parallel particle mixing . . . 11
3.2.2 Review: k-d tree pairing . . . 12
3.2.3 New particle pairing method - sub-volume searches . . . 12
3.2.4 Alternative particle matching algorithm . . . 14
3.2.5 Inter-processor communication of particle data . . . 14
3.3 Load balancing . . . 16
3.4 Statistic tools . . . 16
3.4.1 Eulerian statistics - visualization . . . 18
3.4.2 Python library for post-processing . . . 18

4 Code and Directory Structure **19**
4.1 Template design structure of particles, clouds, and submodels . . . 19
4.2 Adding new sub-models . . . 21

5 Validation **22**
5.1 Shear layer test case . . . 22
5.1.1 Results . . . 22
5.2 Shear layer test case with particles - mmcSprayFoam . . . 24
5.3 Sooting Sandia flame test case . . . 26
5.4 DME G' Flame . . . 29
5.4.1 Results . . . 29
5.5 Performance comparison . . . 29
5.5.1 Parallel scaling of mmcFoam . . . 31

1 Introduction

The simulation of reacting multiphase flows must account for processes such as transport, mixing, chemistry, phase transitions and all their interactions. Particular challenges arise in the presence of turbulence as all relevant scales can no longer be resolved and only the larger scales can be computed directly. Then, all subgrid scale processes including their interactions require modeling. To reduce the high computational costs associated with dense particle methods, a new sparse particle approach called multiple mapping conditioning (MMC) has been developed [2]. A reduction of the number of stochastic particles by a factor of around 100 (and sometimes even more [2, 7]) can be realized due to a conditioning of the mixing operator (that emulates molecular and turbulent mixing) on a reference field. Even though the computational time can be significantly reduced compared to a dense particle approach, the current implementation of the MMC model in OpenFOAM, mmcFoam, is limited in its scalability and applicability to large cases. Further, it is based on the OpenFOAM version 5.x from 2017 and therefore did not benefit from the improvements in numerics and parallel computation of the recent years. Another issue is usability and ease to extend the library. Installing and extending the mmcFoam library with additional models should be as easy as possible, to encourage researchers new to mmcFoam to contribute with own models and to elevate the overall code quality. To achieve these goals, following points are addressed:

- Porting mmcFoam from OpenFOAM 5.x to v2306
- Improve run-time performance (single core)
- Improve parallel scalability
- Allow any decomposition method and mesh design
- General improvements considering coding style and testing

1.1 MMC Modeling

The main challenge of simulating turbulent reactive flows stems from the closure of the averaged chemical source term, $\overline{\dot{w}}$, required for the species transport equation,

$$\frac{\partial \bar{\rho}\widetilde{Y_k}}{\partial t} + \nabla \cdot \left(\bar{\rho}\widetilde{Y_k}\widetilde{\boldsymbol{u}}\right) = \nabla \cdot \left(\Gamma_k \nabla \widetilde{Y_k}\right) + \overline{\dot{w}_k}. \tag{1.1}$$

Since the reaction rate is highly non-linear, the average reaction rate cannot be computed with the average quantities of temperature and mass fraction,

$$\overline{\dot{w}_k(T,Y_k)} \neq \dot{w}(\overline{T},\overline{Y_k}). \tag{1.2}$$

Over the years several models have been proposed to close the chemical source term. One modeling framework that introduces no simplifications for the chemical solution is the transport of a joint probability density function (JPDF) of the reactive scalars. Typically this is implemented as a particle based Monte Carlo method, where stochastic

particles emulate the evolution of the reactive scalars. This advantage is, however, bought by having to model the molecular diffusion, or mixing of the stochastic particles. In a straight forward implementation a sufficient number of particles, around 100, must be present in each computational finite-volume cell. To model the molecular diffusion, particle pairs close in physical space are selected and mixed with a suitable mixing model, such as the Curl's coalescence/dispersion model [4, 3]. The high particle number density required for this approach significantly increases the computational costs, making large scale simulations very expensive or even unfeasible.
Recently a new method, based on multiple mapping conditioning, has emerged, which significantly reduces the amount of stochastic particles required by several magnitudes [2, 7]. The concept behind this method is to base the selection of the unique particle pairs not only on the distance in physical space, but condition it with an additional reference variable. For non-premixed or diffusion flames the mixture fraction is typically chosen. Hence, for a one dimensional reference space the squared distance between a particle p and q is expressed with,

$$d^2_{(p,q)} = \left[\sum_{i=1}^{3}\left(\frac{d^{p,q}_{x_i}}{r_m}\right)^2\right] + \left(\frac{d^{p,q}_{\xi}}{\xi_m}\right)^2 . \tag{1.3}$$

Here, r_m and f_m are characteristic lengths of the physical and reference space respectively. Conditioning with the reference variable prevents particle mixing over the flame front, hence for a large mixture fraction distance, as illustrated in Figure 1.1. Here particle p1 and p2 are close in physical space but are separated by the flame front and thus have a large distance in reference space. In contrast, particle p3 is further away but within the same range of mixture fraction space, hence p1 will mix with p3 instead of p2, preventing the mixing over the flame front.

1.2 Structure of the report

The report is separated into chapters. Chapter 2 describes the necessary main changes in the code to link the existing library with OpenFOAM v2306. The third chapter describes changes implemented to increase the performance and usability of mmcFoam. In the following chapter 4 the code structure is described, starting from the directory level, to how to add a new sub-model. In the last chapter the new mmcFoam library is validated using two test cases.

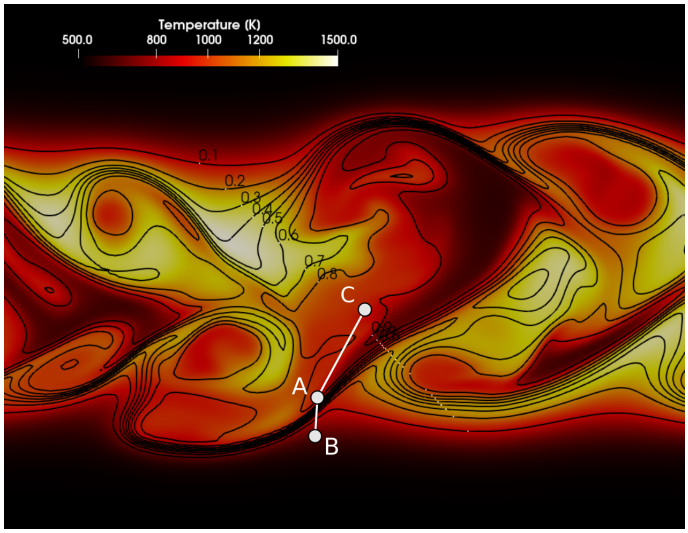


Figure 1.1: Sketch of particle pairing based on multiple mapping conditioning. Despite the small physical distance between particle A and B, particle A will mix with the more distant particle C due to the space in mixture fraction space. The background image is the temperature profile of a double shear layer case, and the black lines represent iso-contours of the mixture fraction.

2 OpenFOAM Changes from 5.x to v2306

The update of OpenFOAM 5.x to the ESI version OpenFOAM v2306 has introduced several changes at critical libraries for mmcFoam. Some of these changes revert the flow of dependencies and have significant effects on the code structure, e.g. the ode solver construction. These issues have to be addressed to allow compilation of mmcFoam with OpenFOAM v2306 without duplicating old 5.x code and keeping an outdated software structure. In the following major points that had to be adapted are presented.

2.1 Particle chemistry models

In mmcFoam 5.x the `mmcThermoPhysicalModels` replicated most of the chemistry model in OpenFOAM. To avoid code duplication and to use the OpenFOAM v2306 chemistry model structure, the mmcFoam chemistry models inherit the standard `StandardChemistryModel` and add the new `particleCalculate()` function to calculate the chemistry on a particle.
From OpenFOAM 5.x to v2306 the design of the ODE class and the integration has changed significantly. In OpenFOAM 5.x the ODE class is derived from the selected chemistryModel, which constructs a new `ReactionThermo` object and stores it internally in an `autoPtr`, see Figure 2.1. This design has the obvious flaw, that if already a thermo model exists, it is duplicated here. This has been changed in OpenFOAM v2306 with `ReactionThermo` model as an input to the `chemistryModel`. As any type of `ReactionThermo` model can be selected for the `chemistryModel`, it is a template parameter, see Figure 2.2. This change in the design of the `chemistryModel` class however, requires changes of the mmcFoam cloud design.

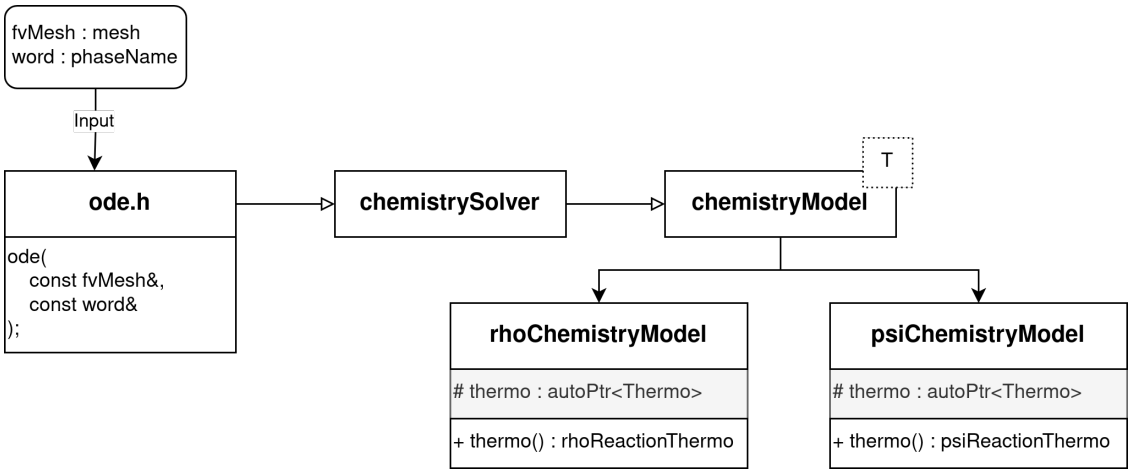


Figure 2.1: OpenFOAM 5.x ODE class integration and linking with the chemistry model.

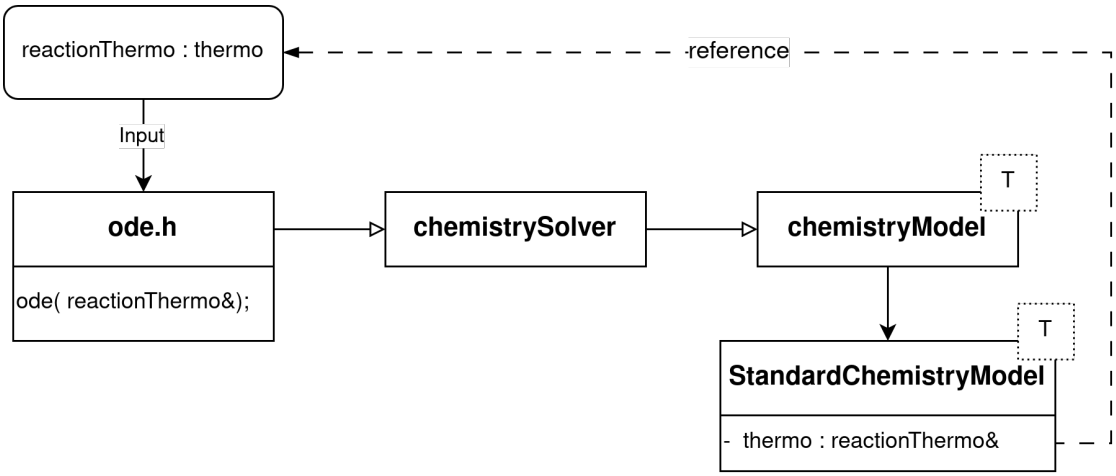
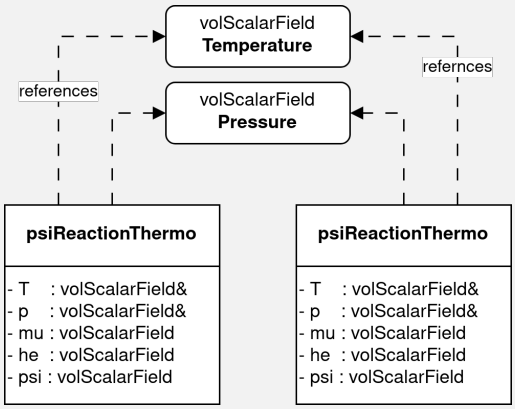


Figure 2.2: OpenFOAM v2306 ODE class integration and linking with the chemistry model. Note, that here the `ReactionThermo` object is references and not created in the `chemistryModel`.

OpenFOAM derives all thermodynamic models from the `basicThermo` class, which stores an internal reference to the temperature and pressure field. It is important to remember that these are references to existing fields and not new fields for each thermo object, see `basicThermo::lookupOrConstruct()` in `basicThermo.C`. Hence, all thermo objects share the same temperature and pressure field. However, enthalpy, species mass fraction, etc., are separate fields.



2.1.1 ODE solver and thermo library

In the previous section it was outlined, that the ODE solver in OpenFOAM v2306 requires now a non-const reference to a thermodynamic model. To replicate a similar behavior as for mmcFoam-5.x, a new thermo model is created in the `ThermoPopeCloud` to pass to the ODE solver, separate from the one in `createFields.H`.
The mmcFoam 5.x version constructs the `psiReactionThermo` object in `createFields.H`, packages this in a `SLGThermo` object and uses it then as an input for the `ThermoPopeCloud`. Unfortunately, the `SLGThermo` class can only return a const reference to the thermo library. Due to the ODE solver now requiring a non-const access to the thermo library, the old code structure is no longer possible. To provide access to the thermo library the `ThermoPopeCloud` now creates and owns a new ReactionThermo object and can provide non-const access. To implement this, a second template parameter, ReactionThermo, is added to the `ThermoPopeCloud` which describes the type of the thermo library used in this cloud, see Figure 2.3. This design allows each created cloud to potentially have different thermo libraries. Currently, `psiReactionThermo` is used for all realizations of the `ThermoPopeCloud`. Further, the cloud provides and owns the `SLGThermo` object, which is required for the composition model or the synthesis sub-models.

Potential Optimization: To improve memory management and avoid the duplication of the internal fields of the thermo model, e.g., viscosity, compressibility, enthalpy and especially the mass fraction fields, `Y`, the `ThermoPopeCloud` could lookup the thermo model in the object registry. In the case of mmcFoam this would mean that the first created `psiReactionThermo` would be used in the `ThermoPopeCloud`, thus providing access to the Eulerian mass fraction fields transported in mmcFoam.

2.2 Particle Tracking

The template tracking data class for each particle was modified to become a non-template class with a template constructor for the cloud. This affects the `calc()` and `move()` functions of particles which now add the cloud to the tracking data as an input.

mmcFoam 5.x

```
template<class ParticleType>
template<class TrackData>
bool Foam::ItoPopeParticle
    <ParticleType>::move
(
    TrackData& td,
    const scalar trackTime
)
```

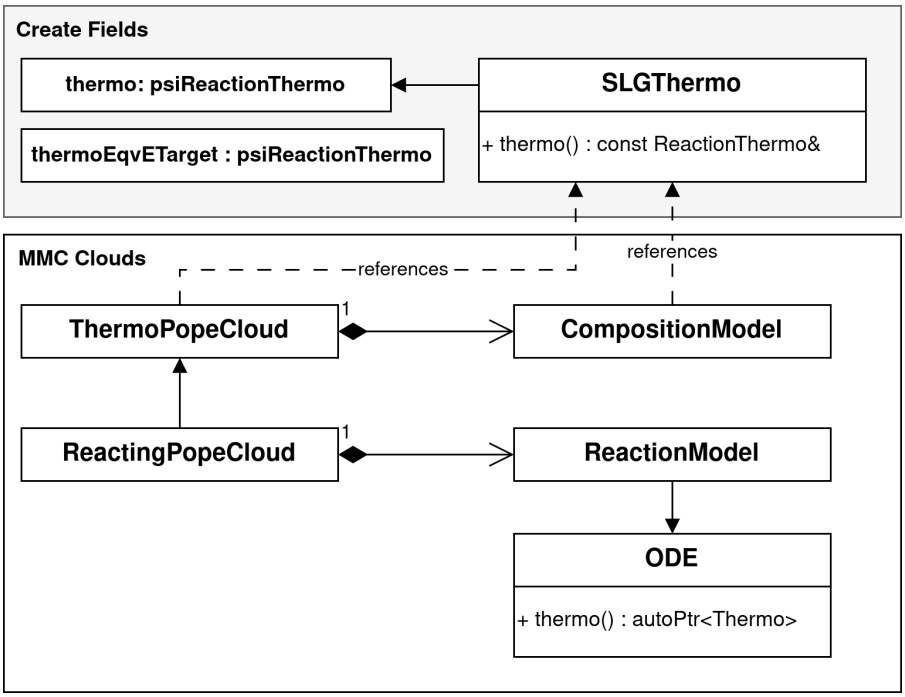
mmcFoam v2012

```
template<class ParticleType>
template<class TrackCloudType>
bool Foam::ItoPopeParticle
    <ParticleType>::move
(
    TrackCloudType& cloud,
    trackingData& td,
    const scalar trackTime
)
```

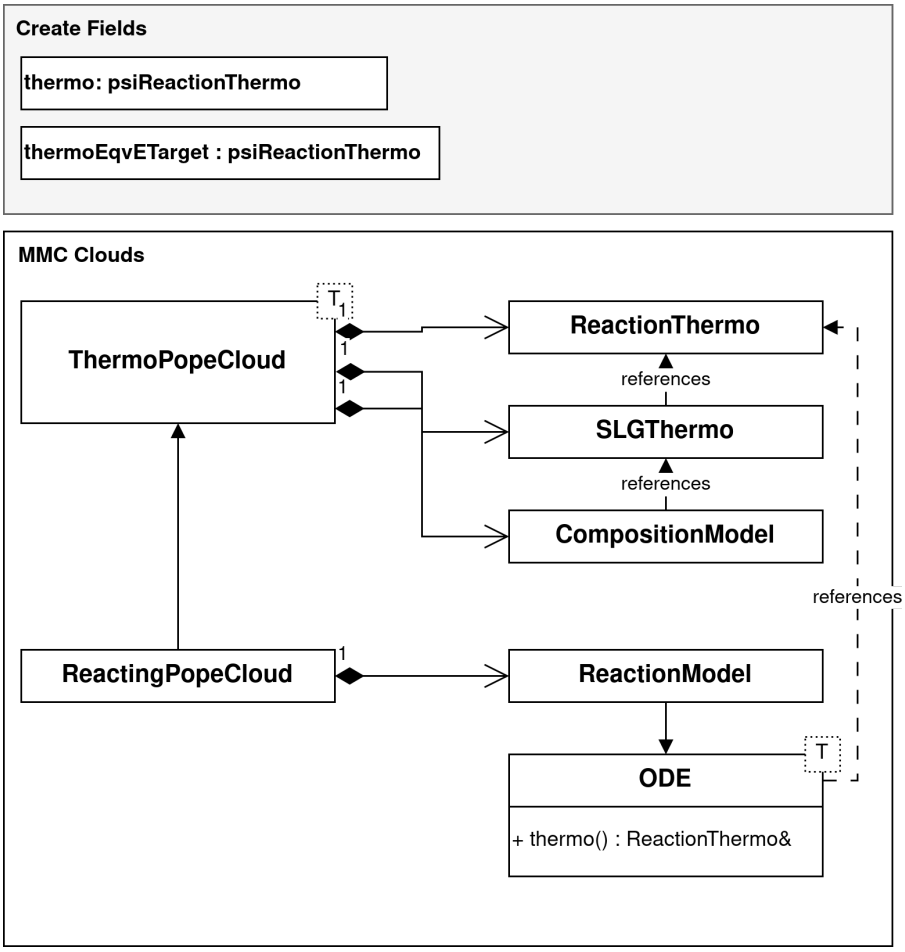
2.3 Other changes

Other minor adaptions from OpenFOAM 5.x to v2306 are:

- Old Random class is deprecated and cachedRandom is now Random.
  See following commits:
  mmcFoam 87d912d1ac78afca2377d367af44d59c24744142
  OpenFOAM b07bc1f0 and 7f0cc004
- Replace the `AllConstIter` with `AllConstIters` which does not require the typename. The old version is causing template deduction errors.
  See commit:
  mmcFoam b4717531a8c66d84b31f37e5fec8ee8f668b4a51
  mmcFoam cb417bba195c1e253294972a82f79bed78c410e8
- The viscosity field is now directly looked up from the thermo library over the object registry. It is no longer required to create a volScalarField mu in createFields
  See commit: 3a23b2c19938645f64f155d26083d1b6aa4ece6a
- Modification of the run time selection mechanism, see also `https://develop.openfoam.com/Development/openfoam/-/wikis/upgrade/v2112-Developer-Upgrade-Guide`
  See commit: e357024266a4672cf33f821cb345e75a0a79d110



(a) mmcFoam 5.x



(b) mmcFoam v2306

Figure 2.3: Integration of the thermo model in mmcFoam for version 5.x (a) and v2306 (b). In the later the thermo models are owned and managed by the thermo cloud. Note, classes are depicted in a simplified manner, only for this aspect important associations are displayed.

3 Performance & Usability Improvements

The mmcFoam 5.x implementation has some restrictions and scalability problems that prevent the usage of mmcFoam on very large scale simulations with high performance computers (HPC). As problem sizes grow and CPU clock rate is physically limited, the only option is to parallelize the code as best as possible to achieve a high overall parallel efficiency. With this goal in mind, bottlenecks are addressed in the following.

3.1 Particle management and super mesh

Particle management is a significant challenge in applying the current mmcFoam software, especially for new users. Due to the sparse particle approach of mmcFoam, it must be ensured that enough notional particles exist in the reaction zone to predict the chemical reaction accurately. This is important, as the temperatures are high within the reaction zone, which leads to a large random walk compared to the cold particles surrounding the reaction zone. Hence, the outflux of particles out of the reaction zone is larger than the influx. If the particle density within the reaction zone is not managed, eventually, too few particles exist, and the model is no longer accurate. This problem is solved in mmcFoam 5.x by creating a second mesh, called superMesh, where each cell represents a control volume in which a given particle number is enforced. Unfortunately, the current implementation with a second mesh restricts the usability and flexibility of the code significantly. Nearly all simulations require parallel execution on multiple cores due to the problem size investigated, which requires decomposing the domain into sub-domains for each processor. With the current implementation, the division into sub-domains is restricted by the design of the super cells, as super cells cannot be distributed to several processors. Hence, during the design of the super mesh, not only the particle number management but also the future decomposition of the mesh must be accounted for, significantly limiting the flexibility and scalability of the code.
To decouple the concept of a superMesh for particle population management from the run-time conditions of the simulation, such as chosen decomposition method, a new `particleNumberController` is created. This class is responsible for initializing the particle numbers in the super cells and enforcing the particle number density defined by the user. The general algorithm does not change from mmcFoam 5.x to v2306, but the concept of a super cell is now abstract and no longer requires a superMesh on the processor. Instead, the superMesh is written with blockMesh to the constant/ folder of the case, and the entire superMesh is read by each processor during a parallel run from the case constant folder. As typically, the superMesh is much smaller than the LES or DNS mesh, it does not create a significant memory or performance issue. Once each processor has read the superMesh, the `particleNumberController` searches for all cells located in the super cells of the own processor. As super cells can now be split over several processors this process requires communication to collect the cell information not locally available on the processor. Further, as some operations, such as collecting all particles in a super cell or getting the field data of a super cell, require processor communication, the class provides functions such that the user does not need to care about the implementation details. The two functions for this are

particleNumberController template functions

```
// Correct the particle number within the super cells
template<class CloudType>
void correct
(
    CloudType& cloud
);

// Get particles of the cloud located in superCellI
template<class CloudType>
DynamicList<typename CloudType::particleType*>
getParticlesInSuperCell
(
    CloudType& cloud,
    const label superCellI
) const;

// Reconstruct the given field for each super cell of the processor
template<class FieldType>
List<DynamicList<typename FieldType::value_type>>
fieldDataInSuperCell(const FieldType& field) const;
```

The conceptional idea behind this is, to separate the high-level, so called business-rules, from the low level implementation. The best-practice would of course be to create an interface class of which then the implementation inherits. Yet, as we only have one particle management and to avoid additional confusion, it is here programmed simpler with only one class to manage the particle number density, see Figure 3.1. If in the future multiple options are made available for the particle management, an interface class combined with run time selection can be implemented.
In the case here, the high-level policy are,

- correct the number of particles within a super cell – `correct()`
- access to all particles within a super cell of a given cloud – `getParticlesInSuperCell()`
- get access to a reconstructed field of data of the super cell – `fieldDataInSuperCell()`

How these policies are achieved are now subject of the implementation. In the current case the number of particles is enforced by an upper and lower bound and super cells are constructed from an OpenFOAM mesh. However, this could now easily be changed to a different set of rules, e.g. that a particle number density is defined based on a function in space which is enforced, instead of discretizing the space into super cells. The only thing the new class needs to provide are the interface functions mentioned above. As a final note, the current `particleNumberController` cannot be a pure virtual interface class with the three functions declared above, because some classes of mmcFoam depend on the concept of the super cells and cannot handle yet an abstract version.

3.1.1 Possible improvements

Currently the total number of particles in each super cell is defined globally with an upper and lower bound. If one of the bounds is reached the particle number is enforced to be the target value. Hence, it does not clip to the upper or lower bound but goes straight to the target value. If a large upper and lower bound is imposed this causes significant changes once the bounds are reached, possibly it would be better to clip to the bounds than to enforce the target.

(a) mmcFoam 5.x

(b) mmcFoam v2306

Figure 3.1: Design of the particle management in the `ItoPopeCloud` for mmcFoam 5.x (a) and the new implementation with a particle number controller (b).

3.2 Particle pairing and mixing

A crucial function of the MMC method is the selection of particles to mix and their mixing speed. Particle pairs have to be selected to minimize the distance $d_{p,q}$. For further details about the theory, see [2, 1]. On a more abstract, mathematical level, the problem described is the assignment problem, or minimum cost bi-partite matching, which can be solved with linear programming. The problem is that n workers have to be assigned to n jobs with an associated cost function, f. Finding unique particle pairs is a special case of an assignment problem with the cost function $f(p,q) = d_{p,q}$, as described in Eq. (1.3). To solve these problems, the cost matrix has to be known, which contains the cost of mixing particle p in row i with particle q in column j. Here an example case with four particles is given. To avoid the pairing of particles with themselves, the distance is set to infinity for $i = j$,

	Particle 1	Particle 2	Particle 3	Particle 4
Particle 1	Inf	**1.0**	2.0	5.0
Particle 2	**1.0**	Inf	0.8	1.3
Particle 3	2.0	0.8	Inf	**1.2**
Particle 4	5.0	1.3	**1.2**	Inf

In the given example, particle 1 would mix with particle 2, and particle 3 would mix with particle 4. This problem can be solved in polynomial time with, e.g., the Hungarian algorithm [6]. However, the main challenge in this case with this set of problems is that the calculation of the cost function is quite expensive, and generating the cost matrix, $0.5n^2$ calculations of the distance $d_{p,q}$ are required, where n is the number of particles. Further, the example shows how a greedy algorithm, which selects at each iteration a locally optimal choice, can lead to extremely bad results. In this example, a greedy algorithm would pick particles 2 and 3 to mix, which leaves 4 and 1 as the second pair, which, however, have a very high cost of 5.0.

To avoid the calculation of the cost function, the constraint to find the minimum pairing configuration is relaxed for an approximate mixing distance, which is close to the optimum. For this, an efficient k-d-tree implementation is used, which divides the particle population until only two particles exist in one tree node. The tree nodes now represent the particle pairs. Only log(n) sort operations are required for this operation, and at no point, the cost function has to be evaluated. Further, the implementation of the k-d tree is straightforward as only the construction of the tree but not the traversing for nearest neighbor search must be considered.

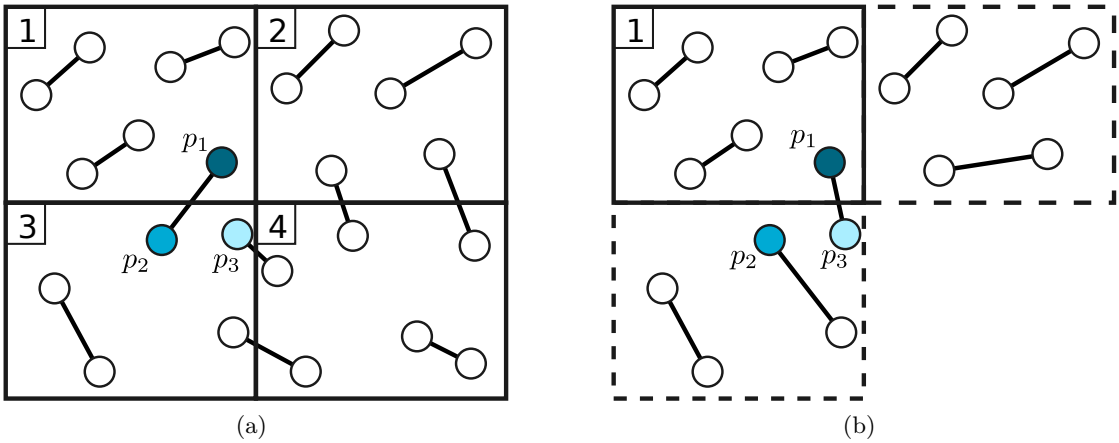


Figure 3.2: Sketch of particle pairing for a case decomposed into four sub-domains, here also referred to as processors. In case (a) all particles in all processors are considered. Case (b) shows the particle pairing if for processor 1 only the particles of processor 2 and 3 are known.

3.2.1 Parallel particle mixing

Most simulations have to be run in parallel on several cores and nodes, to solve the problem in a reasonable amount of time. To run the simulation in parallel, the problem, here the physical domain, is divided into smaller sub-domains and distributed to the cores of the CPU, such that each core solves for one sub-domain. As each sub-domain has its own memory, information must be explicitly exchanged via Message Passing Interface (MPI) commands. Considering now the particle mixing and pairing, it means that the particles are distributed to the individual cores and do not know about the others by default. Figure 3.2a shows a sketch of particle pairing for a domain decomposed into four sub-domains in the following, also called processors. As long as all processors know about all the other particle positions, the pairing computed individually on each processor is the same. However, this changes if only a subset of particle positions is known. E.g., processor 1 does not need to mix with any particle on processor 4; However, if now we communicate only the information of processor 2 and 3 to processor 1, the pairing changes, see Fig. 3.2b. Hence, a change in the set of particles results in a different matching, even if the missing particles had no part in the pairing of the processor considered.
Now, the question is if we can allow different pairings for each processor or if it violates the MMC assumptions. One important rule is that particles do not mix twice within one mixing step. In the case of a parallel run, this issue would occur if the same particle mixes with two different particles on two different processors. As the processors have no direct access to particle data located on different processors, only copies of the particles can be transferred. This means that in the given example case of Fig. 3.2b, processor 1 has copies of the particles located and stored on processors 2 and 3. Hence if on processor 1, the dark blue particle, p_1, mixes with a light blue particle p_3 instead of p_2 as it would in the fully reconstructed case of Figure 3.2a, it does not affect the mixing for the other processors two, three, and four. Therefore, no other particle data is affected by the different mixing partners, which at first appears as a favorable case. However, this leads to problems for the mass balance. As now different particles mix on different processors, depending on their choice of the subset of particles considered, the species mass fraction

of the new ensemble of all particles does no longer match,

$$\sum_p \sum_i (Y_i)_p \, m_p \neq \sum_p \sum_i (Y_i)_{p,\text{mixed}} \, m_p. \tag{3.1}$$

As particle pairing depends on the choice of the particle set, all processors must always know about all other particle positions to ensure mass conservation. However, this so called global mixing, requires an all-to-all MPI communication which destroys any parallel scaling and has to be addressed before large simulations can be performed on HPC systems with hundreds or thousands of cores.

3.2.2 Review: k-d tree pairing

In the introduction is was outlined, that finding a minimum-cost pairing of particles is computationally too expensive. As described above, to avoid the computation of the particle-to-particle distance with Eq. (1.3), a k-d tree is constructed until only two or maximum three particles are within one leaf node. The principal of the k-d tree is to find in each iteration the dimension with the largest spread, sort the particles according to this dimension and then to split the particle population in half. The described step is then repeated on the new sub-list of particles until only two particles are left, and then the algorithm terminates, see Figure 3.3. It is important to note, that this pairing algorithm will, in most cases, introduce mixing barriers over which no particles will mix! This behavior is described in the following with a simple example. Assume a mixing layer case which consists of a 3D domain with the length, width, and height of $L = 10\,\text{mm}, W = 5\,\text{mm}, H = 5\,\text{mm}$. Typical normalization parameters r_m and ξ_m (see Eq. (1.3) are,

$$\begin{aligned} r_m &= 0.22\,\text{mm} \\ \xi_m &= 0.03 \end{aligned}$$

with this, the normalized coordinates are then,

$$\begin{aligned} x^* &= 45.45 \\ y^* &= 22.73 \\ z^* &= 22.73 \\ \xi^* &= 33.33 \end{aligned}$$

Hence in nearly all cases, the k-d tree will split the particle population first in the physical x dimension. This is illustrated in Figure 3.3, which sketches the k-d tree construction. In conclusion, this means that no particle on the left hand side with a low x-coordinate will mix with a particle on the right hand side with a larger x-coordinate, effectively implementing a mixing barrier. This behavior gets worse if the physical extent of the considered simulation domain is large or severely elongated in one direction, as this will lead to several splits in the same coordinate direction.

3.2.3 New particle pairing method - sub-volume searches

The behavior of the k-d tree algorithm motivates a new pairing method for parallel simulations. As the k-d tree implementation can introduce mixing barriers in the simulation, the condition that all particles have to be considered for pairing can be relaxed. Hence, a new model is proposed, in which the physical domain is split up into sub-volumes in which the particle pairing will be computed. This avoids the inter-processor communication and

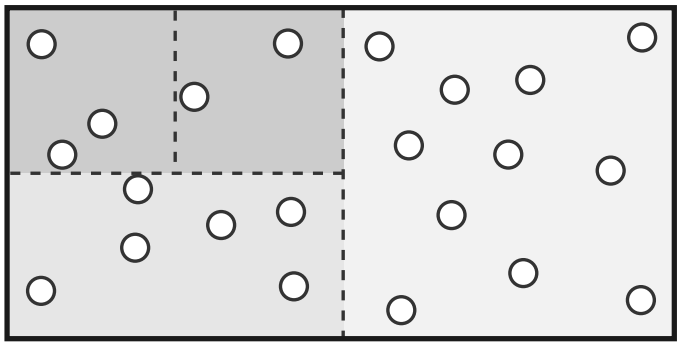


Figure 3.3: Sketch of first three iterations of the k-d tree pairing algorithms splitting the domain. Due to the first split in the longer dimension, the particle pair found in the leaf node on the left side, box II, is selected even though a closer particle for particle A exists on the right hand side of the first split, marked as particle B, hence, preventing any mixing of particles over the barrier marked as a red line.

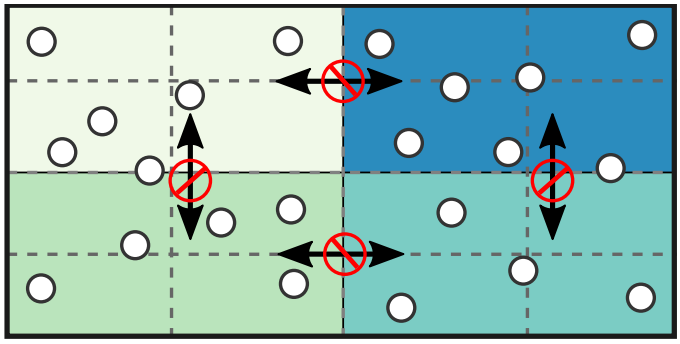


Figure 3.4: Sketch of particle pairing within sub-volumes. The gray lines denote processor boundaries and the background colors the different sub-volumes, consisting of several processor domains. There is no particle pairing, thus mixing, over sub-volume boundaries. Note, that the sub-volume set changes in each iteration and thus mixing barriers are not persistent in time.

double mixing problem described in Section 3.2.1, as only particles within sub-volumes are considered and no particle mixing over sub-volume boundaries is allowed. This idea is sketched in Figure 3.4. The challenge is now to find these sub-volumes and to avoid the introduction of new physical mixing barriers. To solve this, not a single set of sub-volumes is computed but a multitude of sub-volume sets from which one set is selected in each iteration at random. Further, to generate random sub-volume sets one processor is selected at random as a starting point from which the sub-volume search is subsequently developed. Therefore, any mixing boundary introduced by the sub-volumes is not fixed in time, and in each time step a different set of particles is considered for mixing. The second parameter is the number of processors considered to form a sub-volume. Based on the concept, that the k-d tree splits the search domain in the physical dimension if,

$$\Delta X \geq \frac{r_m}{\xi_m}, \tag{3.2}$$

a sufficient number of processors is reached once their extent in one physical dimension satisfies the condition of Eq. (3.2). Hence, adding more processors to the sub-volume once the condition of Eq. (3.2) is satisfied, does not add more particles as potential

mixing candidates, as the k-d tree algorithm will split the particle population first, and always, in the same physical dimension.

3.2.4 Alternative particle matching algorithm

In Sec. 3.2.2 the k-d tree algorithm has been reviewed. However, this algorithm does neither guarantee that the optimal particle partners are found, nor that the maximum mixing distance between particles is lower than a defined threshold. Due to these limitations an alternative particle matching algorithm, based on simple graph-theory, is presented here.

Algorithm description Assume a fully connected, undirected graph $G(V,E)$ with the vertices V and edges E. As we get only the information of the vertices V from mmcFoam (here V are the particles), the edges E have to be computed. The computation of the edges and their edge cost w_e is however quite expensive. Therefore, to reduce cost a modified graph $G^*(V,E^*)$ is proposed where, E^* is computed by finding the m nearest neighbors to each vertex v. This can be done by a k-d tree, HNSW algorithm or similar. The algorithm is as follows:

a) Generate the reduced graph $G^*(V,E^*)$ from the vertex list V with a suitable algorithm, e.g., k-d tree or HNSW.

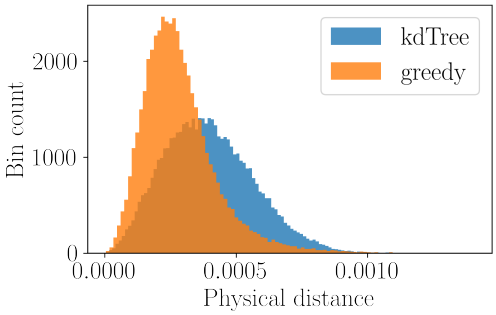
b) Sort all edges in E in ascending order in a list L_E

c) Loop over the sorted edges in L_E and check if the edge weight is less than the maximum mixing distance, $w_e < l_{\max}$. If it is smaller than $l_{\max}$ and the vertices (p,q) connected by e are not yet part of the matching M, $(p,q) \notin M$, add the matching (p,q) to M.

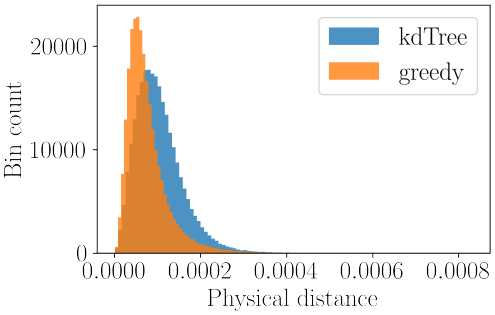
Comparison to k-d tree The distances found with the newly proposed algorithm is compared to the current k-d tree implementation first for a set of 1×10^5 uniformly distributed random points in space and reference space, and secondly for a real particle data set taken from a double shear layer case.
The results show, that the new greedy algorithm results in significantly smaller mixing distances in physical and reference space for both cases. However, the algorithm is currently about 100 times slower due to the construction of the reduced graph $G^*(V,E^*)$ with the k-d tree. If there would be a faster way to generate the graph, this algorithm should be preferred.

3.2.5 Inter-processor communication of particle data

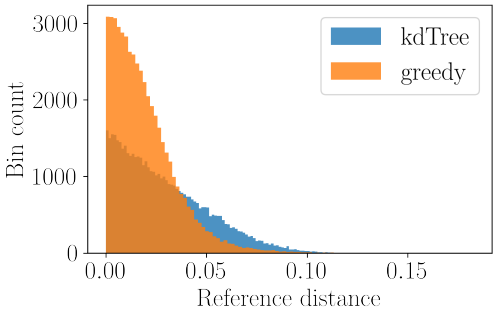
Interprocessor communication is the bottleneck for the mixing model's performance. Therefore, instead of sending all the particle information, a reduced data set of only required properties is sent. The previous mmcFoam version used so-called TPDF fields as a container. However, these fields mimicked the same structure as the particles and did not allow additional access to fields that the particles did not have. This increased the complexity of the code as now changes in the particle classes had to be mirrored in the TPDF field classes as well. To improve this, a new simple base class called `basePa-rticleDataContainer` is created, which stores a list of scalars, labels, and scalar fields. Further, it deals with all IO operations required for parallel handling. If now a class in the mmcFoam code has to send some particle data between processors, but does not wish to send the complete particle, the data to be send can be stored in this container.



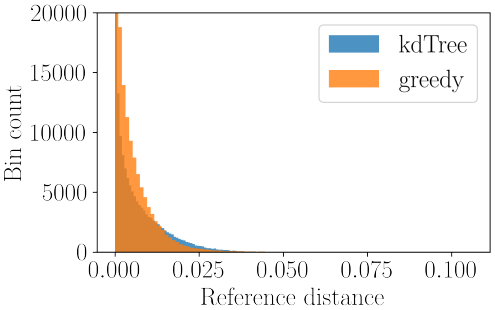
(a) random particle set - pyhsical distance



(b) real particle set - physical distance



(c) random particle set - ref. distance



(d) real particle set - ref. distance

Figure 3.5: Histogram of the physical distance of all pairs found for the greedy algorithm in yellow and the k-d tree in blue. Left image is for the random data set and right for the particle data set taken from the shear layer case.

The procedure to use this class is then as follows. Within the class to use this container, e.g., the `mixParticleModel`, a derived class of `baseParticleDataContainer` is created to allow named access to the fields. The known stream operators can be used to send the container to other processors via, e.g., PstreamBuffers.

Listing 3.1: Usage of base

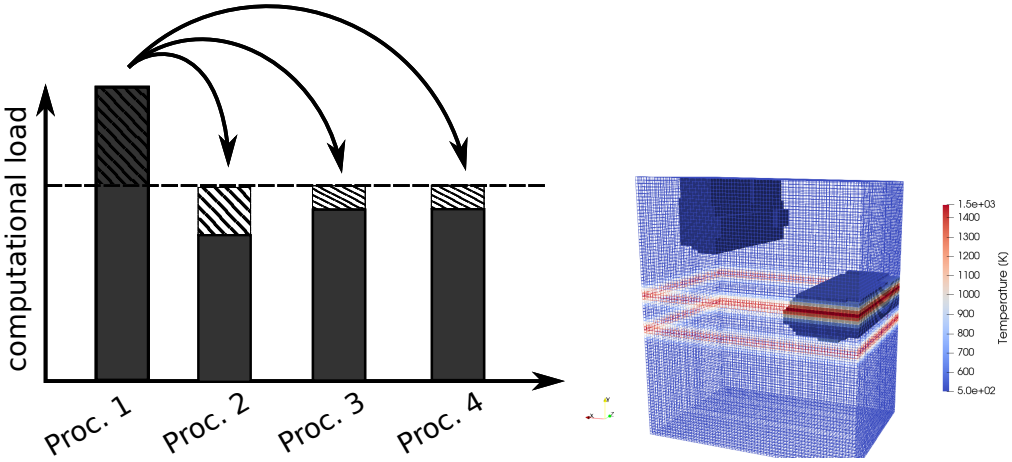
```
class myParticleContainer
: public baseParticleDataContainer
{
    public:
        // Create container with
        // - 2 label entries
        // - 3 scalar entries
        // - 1 scalar field
        myParticleContainer()
        : baseParticleDataContainer<2,3,1>
        {};

        // Allow easy access with named function

        // e.g. first label field
        inline label& myLabelEntry() {return labelValues_[0];}

        // access to a scalar field
        inline Field<scalar>& myfield() {return scalarFields_[0];}
}
```

3.3 Load balancing

Solving the ODE of the reaction is a computationally expensive process, and especially for cases with larger reaction mechanism, it takes a significant portion of the overall required computational time. Unfortunately, the processors in which the flame is currently located have a much higher load than processors in the cold region. Hence, the load between the processors is unequally distributed, leading to poor parallel efficiency, see Figure . A model based on the DLBFoam approach is implemented to achieve a higher load balance. To distribute the load efficiently, the model stores the CPU time required for each particle and sums them up for each processor. The processors are then ranked by their overall CPU time needed to solve the particles. The percentage overheads of the CPU time compared to the overall mean are calculated. In contrast to the old model, not just the highest and lowest ranked processor are matched, but the load of a processor is distributed such that each processor has the mean load, see Figure 3.6.
To solve the reaction, only the enthalpy, pressure, temperature, and species mass fractions are required. Therefore, not the entire particles but only the required data fields are sent to other processors for computation, with the new `baseParticleDataContainer`. For a shear layer test case and 24 core decomposition, the parallel efficiency increases from 66% to 90% once the load balancing is switched on. The load balancing does not reach 100% parallel efficiency, as the CPU times for each particle vary in every time step, and particles are redistributed based on their CPU time required for the last time step.

3.4 Statistic tools

To post-process the results of mmcFoam two tools to gather the particle statistics are provided by the library. The first tool, called `eulerianStatistics` maps all particles in



(a) (b)

Figure 3.6: (a) Sketch of load balance to solve the chemical reaction for different processors and their overheads. (b) Scotch decomposition of a shear layer case, showing two processors, once located in the hot flame and second outside in the cold bypassing flow.

(a) no load balance (b) load balance

Figure 3.7: POP metrics for a shear layer cases with and without load balancing. POP metrics are calculated with scalasca and scorep.

each time step to a 2D plane and averages the results in time. The second one, called `particleStatistics`, samples the particle data within defined probe areas and writes these out for a given sample interval. Both tools can now handle any decomposition of the mesh and writing out the files for parallel cases. Note, that it is not possible to write with multiple processors into the same file! The order of information being written depends on the operating system, the file system used and the hardware. Potentially special functions like MPI_FILE_WRITE could be used but require special handling during compilation, see File IO (HLRS). Currently, a class handler called parallelFileIO is used which collects the information from all processors and then writes only with the master processor to the file. For detailed description on the control and usage of these two tools over the dictionary see the code header files.

3.4.1 Eulerian statistics - visualization

The Eulerian statistics tool maps the particle data for each sample interval to a given 2D plane and averages the results over time. The 2D plane is generated with the OpenFOAM standard `sampledPlane` class, which cuts the mesh to generate the plane. Hence, the plane has the same mesh structure as the underlying base mesh. This is the reason why it is called Eulerian average, because the particle data is binned and averaged to the cell size of the mesh, not because Eulerian fields are sampled. The results are written out when a time step write event is called. This is implemented by making the Eulerian statistics tool a `regIOObject`. To make it easier to visualize the results, next to a file containing the values and position data in OpenFOAM lists, the averaged results are written as VTP files which can be directly loaded into paraview.

3.4.2 Python library for post-processing

To make reading and post-processing of the written Eulerian or particle statistic data easier, a mmcFoam python library called mmcFoamReader is provided with mmcFoam. This library provides tools to load the particle or Eulerian sampled data into python and to plot the results.

4 Code and Directory Structure

The software directory of mmcFoam v2306 is structured as follows,

```
mmcFoam
├── applications
├── docs
├── Doxygen
├── etc
├── notes
├── python
├── src
│   ├── lagrangian
│   │   ├── mmc
│   │   └── spray
│   ├── mmcChemistryModel
│   └── mmcSupport
├── tests
├── thirdParty
└── tutorials
```

The structure follows the general OpenFOAM design of an applications folder containing the solvers and utilities, a src folder which contains all libraries, tutorials with simple test setups, and a thirdParty folder for libraries not included or written in the framework of OpenFOAM. Here, also some additional folders for testing and documentation, such as docs for mkDocs, or Doxygen are present. The folder notes contains mostly outdated PDF files of the solver and code design prior to the upgrade to v2306. This folder will probably be deleted in the future. The last folder is the etc folder containing the environment variables for the build and installation of the software.
The src directory contains all mmcFoam libraries, separated into three categories,

lagrangian All libraries for handling the particles in mmcFoam. This includes the stochastic particles as well as normal spray particles.

mmcChemistryModel Library to solve the chemistry on a particle level, not on Eulerian grids.

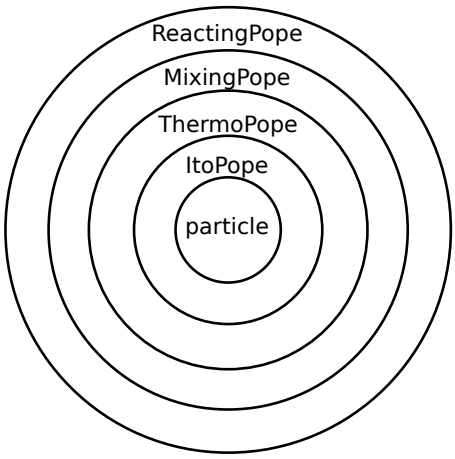
mmcSupport Support libraries for mmcFoam, such as tools to gather statistical data or helper classes required for the other libraries.

4.1 Template design structure of particles, clouds, and submodels

The particles, clouds, and submodels of the mmcFoam library follow the same template design, as the Lagrangian models in OpenFOAM. This design allows to easily create custom particle and cloud classes, by combining the different template classes. Hereby, the design follows a layer principle, which starts with the particles and then goes to the clouds. E.g, a reacting particle including particle mixing, thermodynamic properties, and stochastic movement can then be created with,

Listing 4.1: Basic Reacting Pope Particle

```
typedef ReactingPopeParticle
<
    MixingPopeParticle
    <
        ThermoPopeParticle
        <
            ItoPopeParticle
            <
                particle
            >
        >
    >
>
basicReactingPopeParticle;
```



Note, that in the above code snippet no particle is created but just a short hand notation to avoid writing all the template parameters. Further, as each particle type is derived from the provided template, the structure can be understood as particle properties building up like layers. If now a new thermo pope particle is created, e.g. for pre-mixed simulations, a new basic reacting pre-mixed particle can be easily created by switching only this layer. All the code files for the other particles will be untouched and only the H file containing the new typedef has to be written.
Typically each particle class is accompanied by a corresponding cloud class, e.g., the ThermoPopeCloud class is responsible for the ThermoPopeParticle. Here, the clouds follow the same template design as the particles. This often leads to the confusion, that the sub-models only have access to their respective particle properties, e.g., that the mixing sub-model only has access to the MixingPopeParticle properties and all derived particle layers. This is however not the case! The sub-models are derived from the cloud and the base Cloud class of OpenFOAM has the top particle template as an input. Hence, in the basicReactingPopeCloud the basicReactingPopeParticle is used.

Listing 4.2: Basic Reacting Pope Cloud

```
typedef ReactingPopeCloud
<
    MixingPopeCloud
    <
        ThermoPopeCloud
        <
            ItoPopeCloud
            <
                Cloud
                <
                    basicReactingPopeParticle
                >
            >,
            psiReactionThermo
        >
    >
>
basicReactingPopeCloud;
```

Therefore, all sub-models which have any of the clouds as an input have access to all properties of the basicReactingPopeParticle. This means, that the mixing sub-models do have access to particle properties stored in the ReactingPopeParticle. However, one should avoid requiring properties form particles in other layers in one's sub-model, as

this introduces invisible dependencies that break the template design of the particles and clouds. E.g., assume a new mixing model requires properties from the reacting particle, then for this model to work, the reacting particle always has to be coupled with the mixing particle. If now a new reacting particle is created, e.g., for a pre-mixed case, and this new particle does not contain the same properties of the standard reacting particle, this would break the sub-model, without giving the user a hint that there was the dependency on a specific particle class. Therefore, if possible, avoid using properties of particles outside of the particle the sub-model is written for.

4.2 Adding new sub-models

To add a new sub-model, it is recommended to start from an existing sub-model, e.g., the MMCCurl sub-model, copy and rename the file as a starting point for the new sub-model. Once the new sub-model is defined the macro to generate this sub-model has to be extended. These macros are found in the `include/` folder in the `popeParticles` directory. Their notation follows following syntax, `make<ParticleType><SubModelName>Models.H`, e.g., for the mixing model it is, `makeMixingPopeParticleMixingModels.H`. However, this does not mean that the sub-model is only working with the in the name specified particle type, but shows on which particle properties it depends. If the sub-model has been extending exiting models the process ends here, however, if a completely new sub-model class is added the calls of the macros in the
`basic<ParticleName>/makeBasic<ParticleName>Submodels.C` has to be adjusted.
The general workflow to add a new sub-model is then:

1. Write new sub-model class in the submodels directory. Preferable by copying and renaming existing models to keep a coherent structure.

2. Add the new sub-model to the respective `make<ParticleType><SubModelName>Models.H` in the `include/` directory of the popeParticles.

3. If it is a new sub-model class that did not exist before also adjust the calls of the macros in the `basic<ParticleName>/makeBasic<ParticleName>Submodels.C` located in the `popeParticles/derived` directory.

5 Validation

The new mmcFoam has introduced significant changes to the code basis and therefore it has to be validated, to confirm its correct function. For the validation three test cases, a double shear layer, double shear layer with particles, and a sooting flame are chosen. These test cases validate the core mmc models, coupling of aerosol particles with mmc and a new two-equation soot model [**Weitao**]. All tests are performed with OpenFOAM v2212 with the mmcFoam-v2212 (same as mmcFoam-v2306) version.

5.1 Shear layer test case

The shear layer test case is based on Vo et al. [9] setup of a double shear layer, see Figure 5.1 . For the validation only case I of the two cases presented in the paper is investigated. The mmcFoam settings used for the test case are listed in Table 5.1. The results of the test are compared to the mmcFoam 5.x results and not to the DNS data of Vo et al. [9]. This is due to significant changes in the mmcFoam versions used for the paper of Vo et al. and the current mmcFoam, e.g., the mixing time scale. The mesh size is quite small with only 290 000 cells, which allows to run the test case within a few minutes.

5.1.1 Results

To validate the results the mean and conditional mean profiles of the temperature are compared. In Figure 5.2 the three cases, mmcFoam-5.x, mmcFoam-v2212, and mmcFoam-v2212 with 64 cores are compared. The first two cases are decomposed into 24 processors and the last case uses 64 cores and scotch decomposition. The comparison shows a good agreement between the three cases. Hence, the new super mesh implementation, particle mixing and load balancing seems to work correctly as all three methods are used. Further, it shows that the mmcFoam v2212 can be decomposed into more processors than super cells.

Table 5.1: mmcFoam and turbulence settings for the double shear layer case

Setting	Value
Turb. model	LES Smagorinsky
Super cells	24 super cells
# of particles per super cell	1512
Particle control	off
Mixing model	MMCcurl
aISO	on
Mean mixing time	on

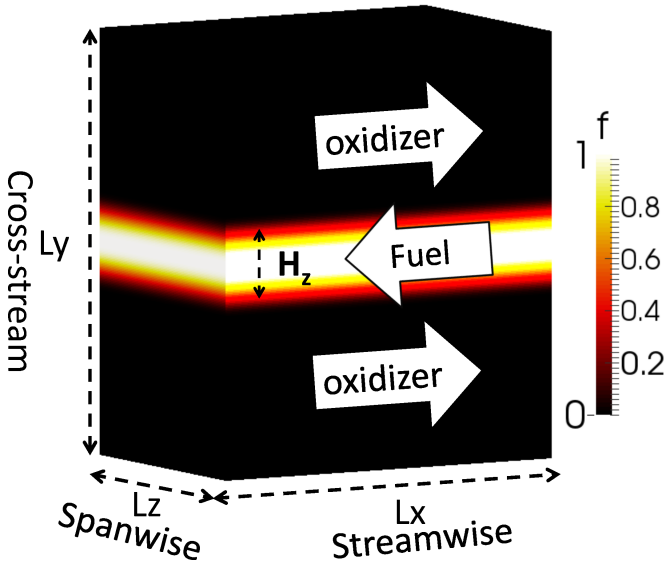


Figure 5.1: Double shear layer setup. Copied from [9]

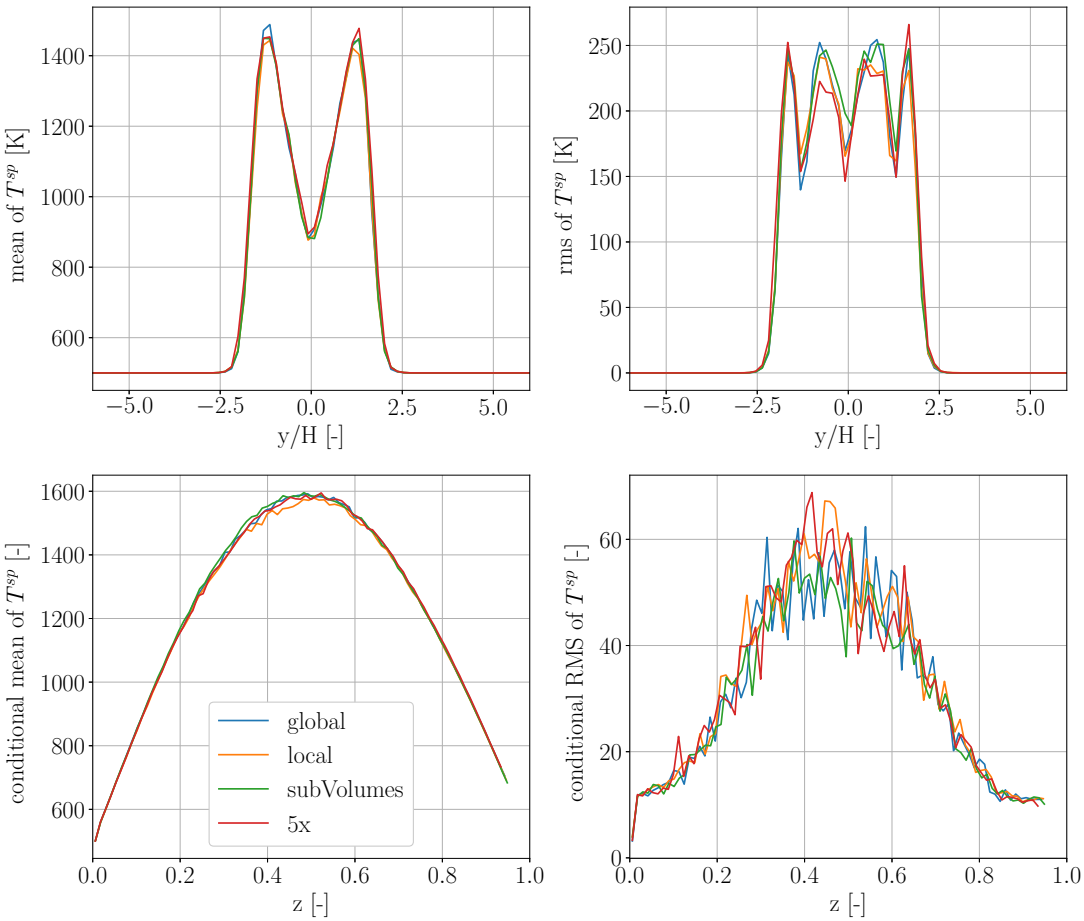


Figure 5.2: Mean temperature and RMS as well as their mixture fraction conditioned values

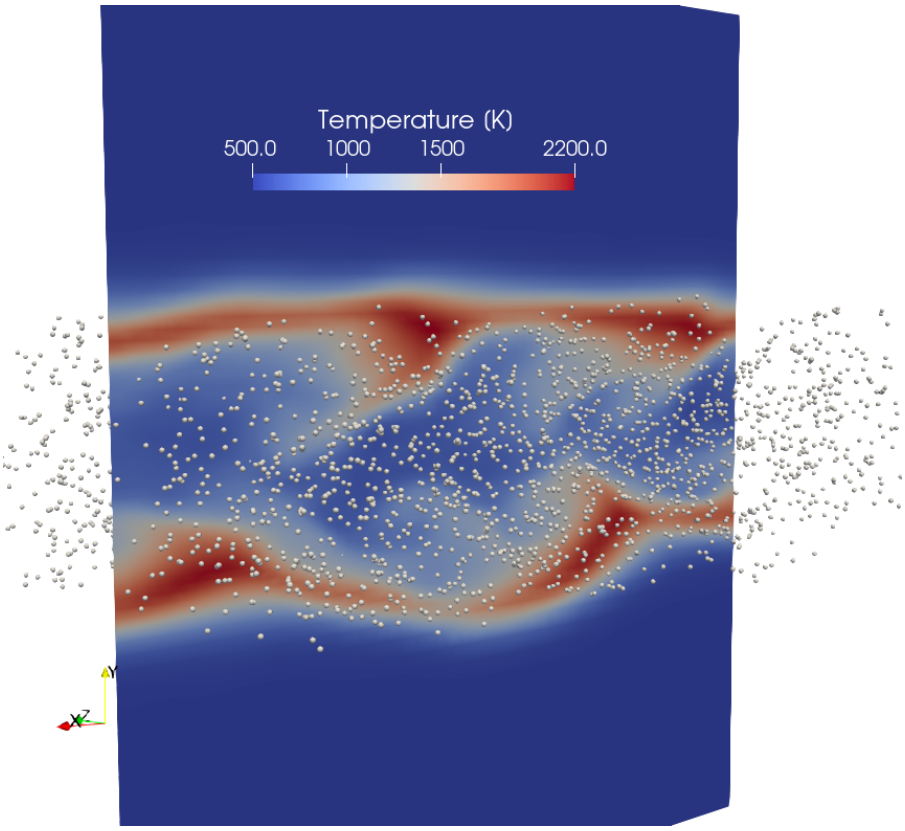


Figure 5.3: 3D image of the double shear layer setup with fuel particles. The color of the slice represents the temperature and the particles represent fuel droplets.

5.2 Shear layer test case with particles - mmcSprayFoam

The coupling of the mmc library with physical particles representing, e.g. droplets, is tested by comparing the mmcSprayFoam results for a double shear layer case. Figure 5.3 shows a 3D image of the case which resembles the setup of Section 5.1, with the addition of fuel droplets in the shear layer.
The comparison of the results of 5.x and v2212 is plotted in Figure 5.3. The results show a good agreement between the two cases. Hence, the spray library of mmcFoam v2212 follows the implementation of 5.x.

> mmcSprayFoam is not a physically validated model. It probably needs to be rewritten and checked before it is applied. See also the work of [5].

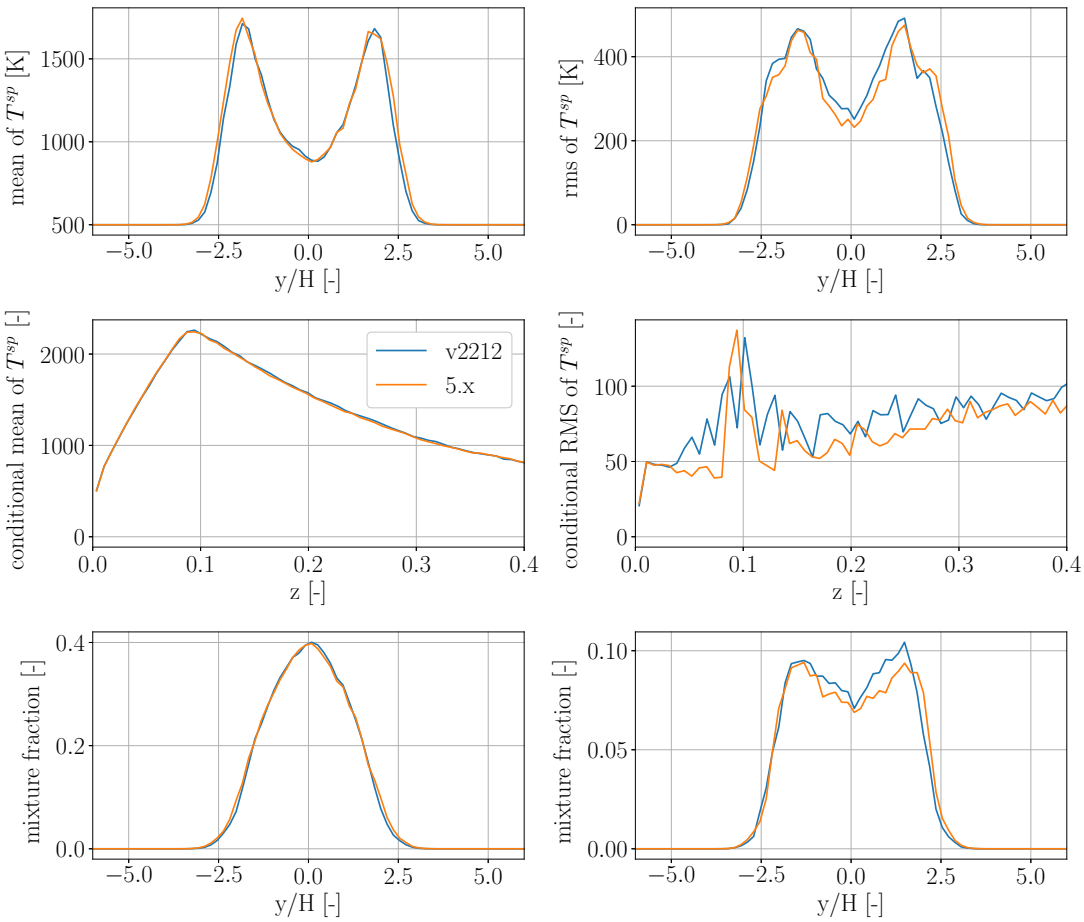


Figure 5.4: Comparison of mmcSprayFoam results for the new v2212 implementation and mmcFoam-5.x.

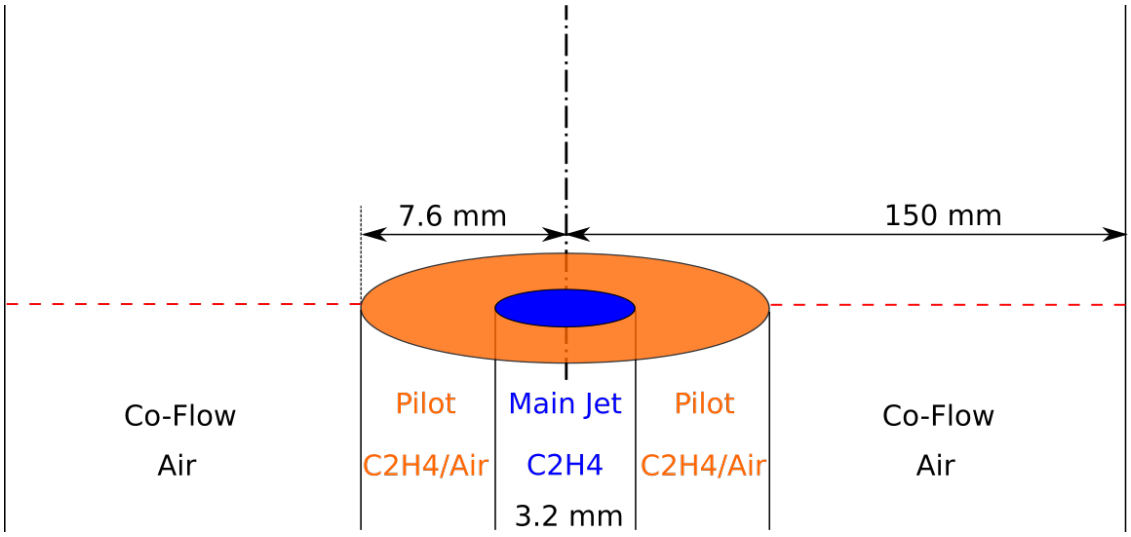


Figure 5.5: Sandia flame case injection conditions.

5.3 Sooting Sandia flame test case

For the second test case a non-premixed, turbulent ethylene flame is investigated. This flame is one of the target flames of the International Sooting Flame Workshop, and has been experimentally investigated at Sandia National Laboratories. The setup consist of a coaxial injector with a central fuel jet of 3.2 mm diameter, surrounded by a pilot stream of hot fuel and air of 15.2 mm diameter. The bulk velocity of the fuel jet is $U_{\text{jet}} = 54.7\,\text{m/s}$, which corresponds to a Reynoldsnumber of about 20 000. The premixed ethylene air mixture of the pilot is at an equivalence ratio of 0.9. The pilot mass flow rate is $1.77 \times 10^{-4}\,\text{kg/s}$, which corresponds to a bulk velocity of $U_{\text{pilot}} = 6.42\,\text{m/s}$ for the given density of $\rho_{\text{pilot}} = 0.1589\,\text{kg/s}$. The pilot temperature is set to the adiabatic equilibrium of 2295.5 K. The surrounding co-flow has a bulk velocity of $U_{\text{air}} = 0.6\,\text{m/s}$ and an ambient temperature of 294 K. The injection conditions are illustrated in Figure 5.5 and the settings are listed again in Table 5.2. To faithfully represent the turbulence of the main jet at injection, a highly resolved pipe is simulated with LES beforehand and the injection conditions sampled. To test the new decomposition and scaling capabilities, the case is run on the super computing system HoreKa on 608 cores with scotch decomposition method.

Figure 5.6 compares the mean temperature profiles and RMS at the two axial positions, $x/D = 134$ and $x/D = 175$, of the mmcFoam 5.x to the v2212 implementation as well as experimental results. The data is collected using the new Eulerian statistics tool and averaging over 0.35 s, corresponding to 22 flow through times. The results show a good agreement of the mmcFoam 5.x results and v2212. Further, they agree well with the experimental results, which however, is rather a validation of the two-equation soot model employed than the mmcFoam solver. Yet, the good agreement of 5.x, v2212 and experimental data suggests that the upgrade to v2212 or v2306 respectively did not add any new major bugs. In addition, this is another validation for the new sub-volume pairing.

The soot prediction of the two-equation model is also agreeing with the experimental and 5.x results, indicating the correct implementation of the new model in mmcFoam v2212, see Figure 5.7.

Table 5.2: Injection conditions of the Sandia flame case

	Velocity [m/s]	Temp. [K]
Jet	54.70	294.0
Pilot	6.42	2295.5
Co-flow	0.60	294.0

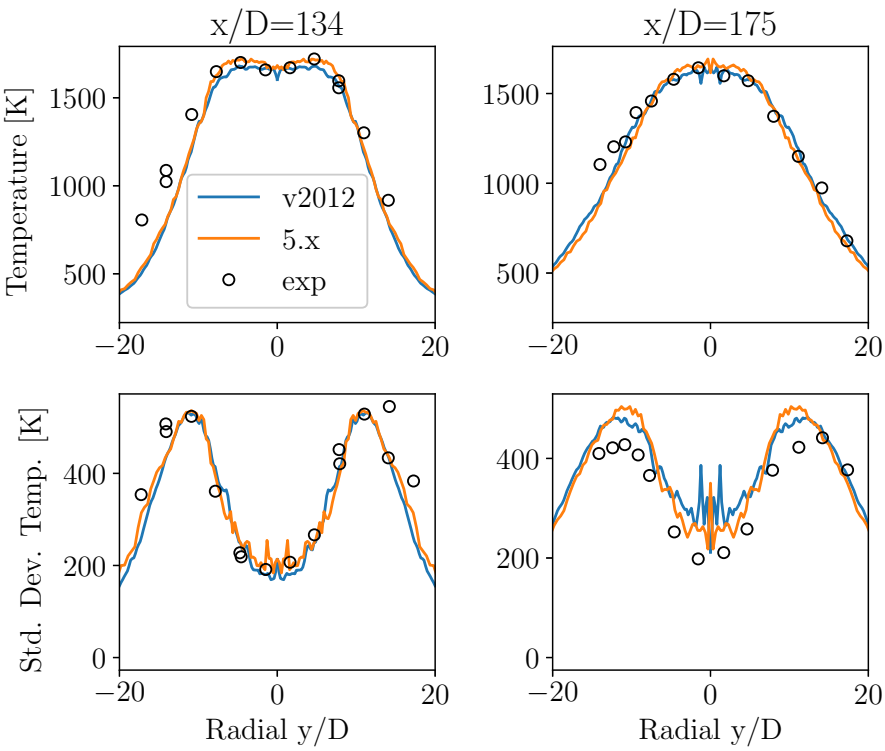


Figure 5.6: Mean radial temperature and standard deviation at different heights compared to experimental results.

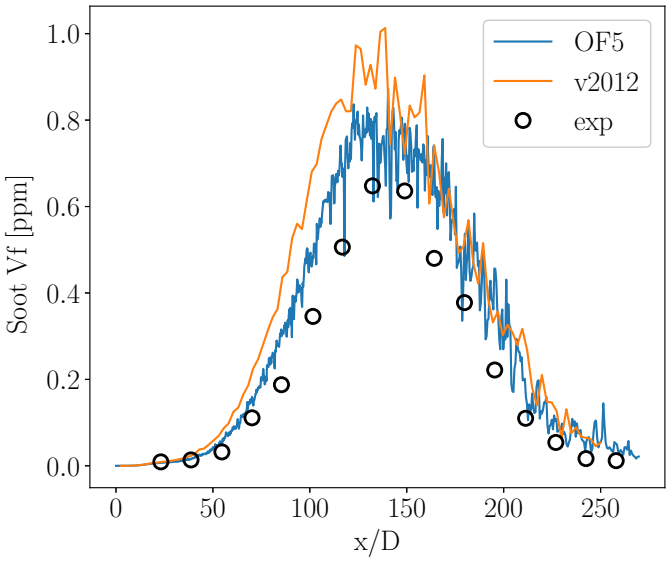


Figure 5.7: Axial distribution of the soot volume fraction of mmcFoam v2212 and 5.x compared to the experimental results (open circles).

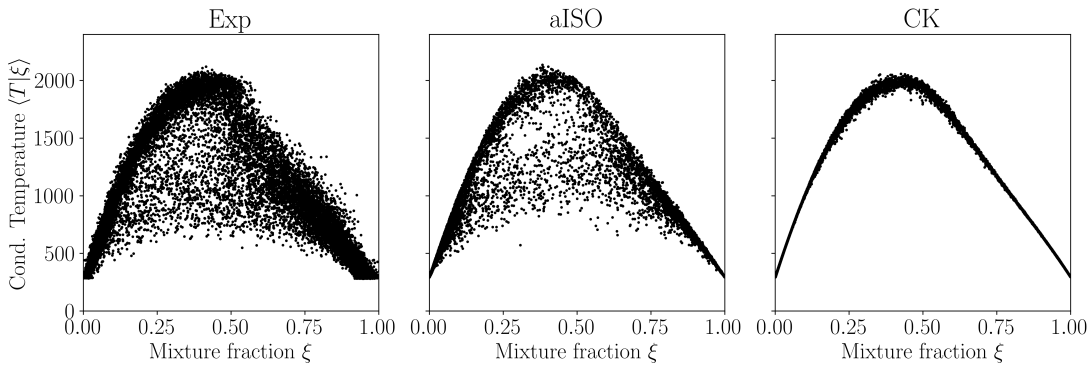


Figure 5.8: Scatter plot of the conditioned temperature over the mixture fraction for the a-ISO and C&K model. Case: DME G' flame.

5.4 DME G' Flame

The dimethyl ether (DME) flame G' of the Sandia DME flame series, is a modified flame G setup with a higher rate of local extinction. The standard configuration of the DME G' flame is a inlet jet with a bulk velocity of 114.75 m/s, jet diameter of $D = 7.45$ mm and a hot pilot gas with 80% of the flame G velocity, $u_p = 12.08$ m/s. This flame has been investigated with mmcFoam 5.x in the work of Neuber et al. [8]. Due to its local extinction, this flame is an excellent candidate to check the mmcFoam v2306 implementation of mixing and particle pairing, as small errors here will lead to global flame extinction.

5.4.1 Results

The results in Figure 5.8 and 5.9 show clearly that the a-ISO model captures the local extinction, whereas the C&K model generates too much mixing, hence preventing the local extinction. This is in agreement with the reference solution in [8] and shows that the mmcFoam-v2306 version can reproduce even challenging simulation conditions.
The results in Figure 5.9 could be averaged longer, however, stability issues would require a more detailed investigation of the DME G' flame setup to allow for a stable longer simulation. The stability issues are also mentioned by Neuber et al. [8]:

> Flame DME G' is quite sensitive to the pilot velocity and by setting the simulation's pilot velocity to 80% of that of flame G, the flame experienced global blow-off. A rather stable solution was obtained in the modelling by increasing the pilot velocity to 85% instead. Even then, the model occasionally produces global extinction due to rare turbulent events. ([8])

Yet, the trend and results downstream match already well, showing that the new mmcFoam model can also solve such difficile problems as the DME G' flame. Further, the results could be obtained in *three* days on a 64 core machine, which is much faster than possible with the previous 5.x version.

5.5 Performance comparison

The changes of the new mmcFoam v2306 version did not only improve usability and flexibility of the library but also improved the single core and parallel efficiency. In Figure 5.10 the run time of the shear layer case of Sec. 5.1 is compared between the old and new

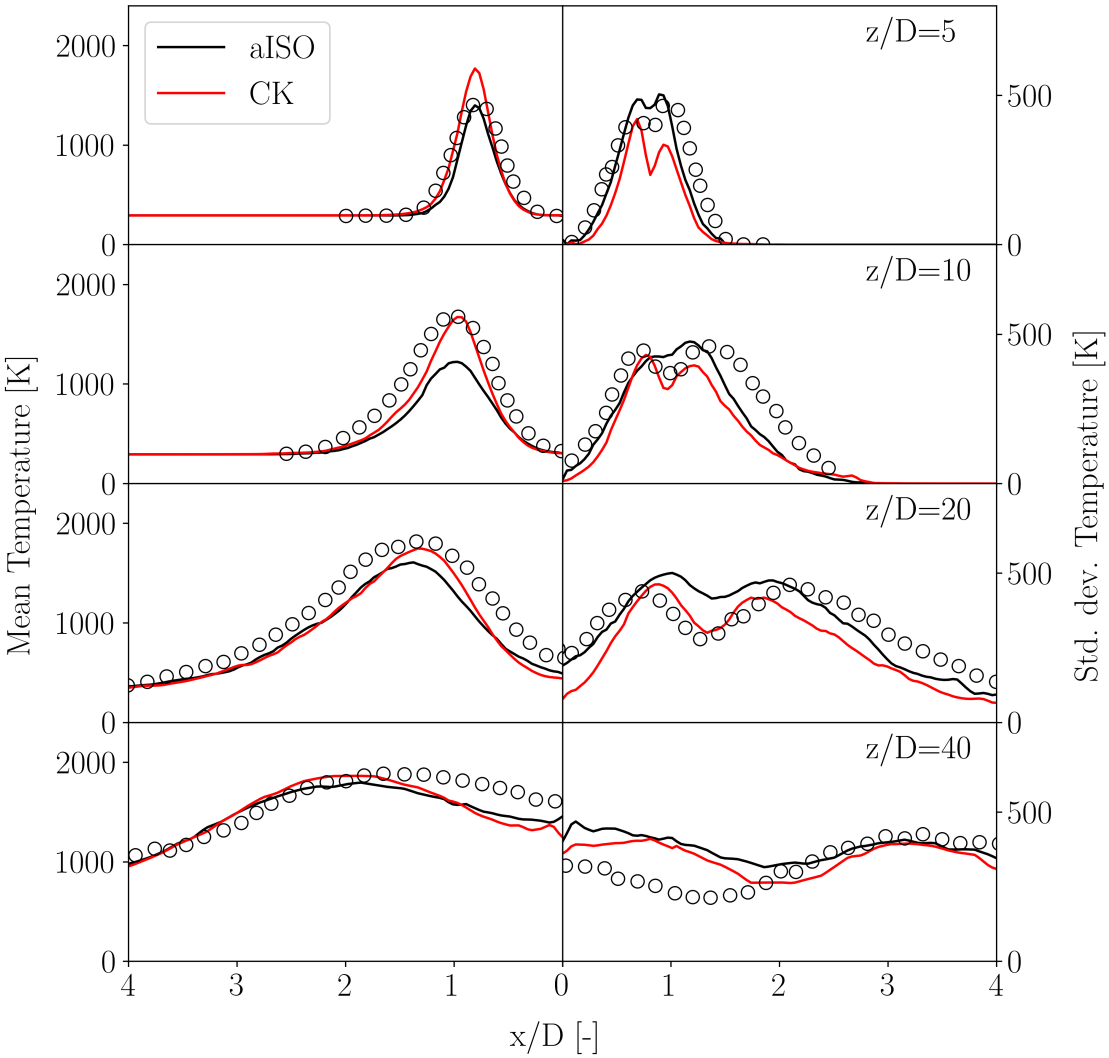


Figure 5.9: Comparison of the mean temperature at different heights for the a-ISO and C&K model. The empty circles are the experimental results.

Table 5.3: Computational setup for the shear layer performance test.

OS	Ubuntu 22.04
Compiler	GCC 9.5.0
MPI	OpenMPI 4.1.2
CPU	2xIntel Xeon Gold 6226

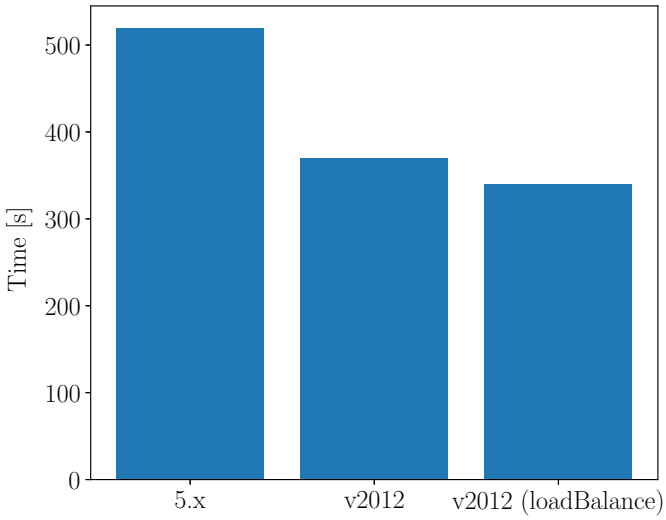


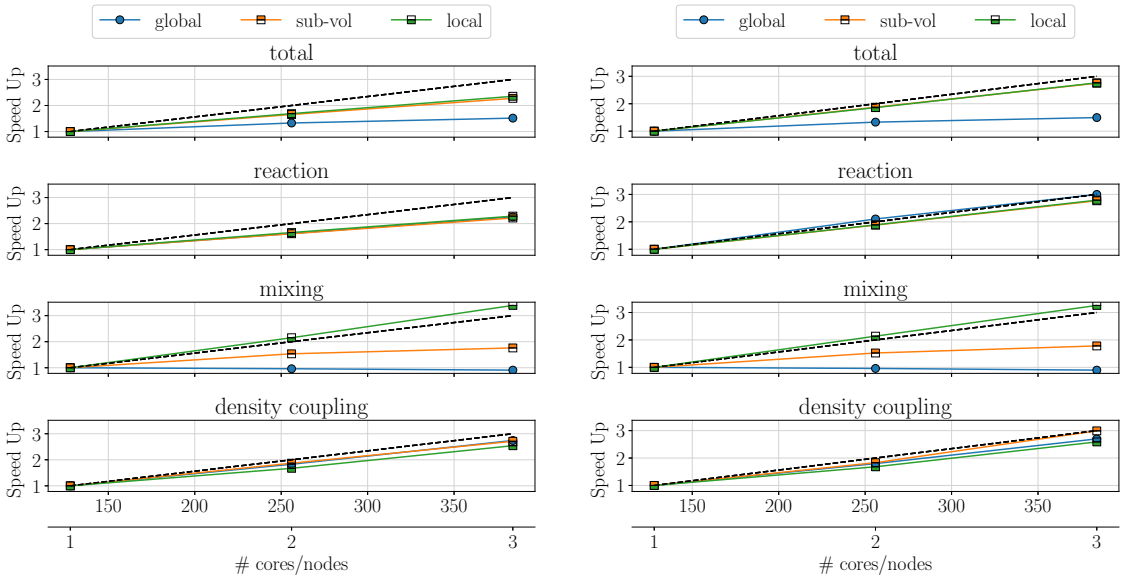
Figure 5.10: CPU time required to solve the shear layer test case on 24 cores.

mmcFoam version. The results show, that the new implementation is 1.4 times faster than the old version and even 1.52 times faster with load balancing enabled. This test used 24 cores, due to the super cell decomposition limit of the old mmcFoam implementation. However, the new mmcFoam library can use much higher decompositions and the parallel scaling is investigated for the shear layer test case with an increased mesh size in the next sub-section. Table 5.1 lists the computational setup for this test case.

5.5.1 Parallel scaling of mmcFoam

The performance of the new mmcFoam solver is tested on the HPC system Hawk of the High-Performance Computing Center (HLRS) Stuttgart. The test case examined is a non-premixed, turbulent ethylene flame, one of the target flames of the International Sooting Flame Workshop, and has been experimentally investigated at Sandia National Laboratories. The mesh comprises of 2.5 million cells and 700 000 stochastic particles. OpenFOAM solvers typically scale well up to a decomposition of 10 000 cells per core, therefore, the solver should scale up to at least 258 cores for this test case. To investigate the effect of the different components of the solver, the scaling is split up into a total scaling of the overall solver, scaling of the particle pairing and mixing, solving the particle reaction on the particles, and coupling the particle densities with the Eulerian field. Figure 5.11 depicts the normalized scaling of the solver, with and without load balancing, and for the three particle pairing methods, global, local, and sub-volumes. The first pairing method is the old implementation of mmcFoam, in which the total particle population has to be known to all processors to find a consistent particle pairing for all. The second option selects particle mixing on the processor only; hence other processors are not considered. This can be a potential option for cases that do not use a high decomposition. However, this requires validation by the user to check if the results can be trusted. The last pairing

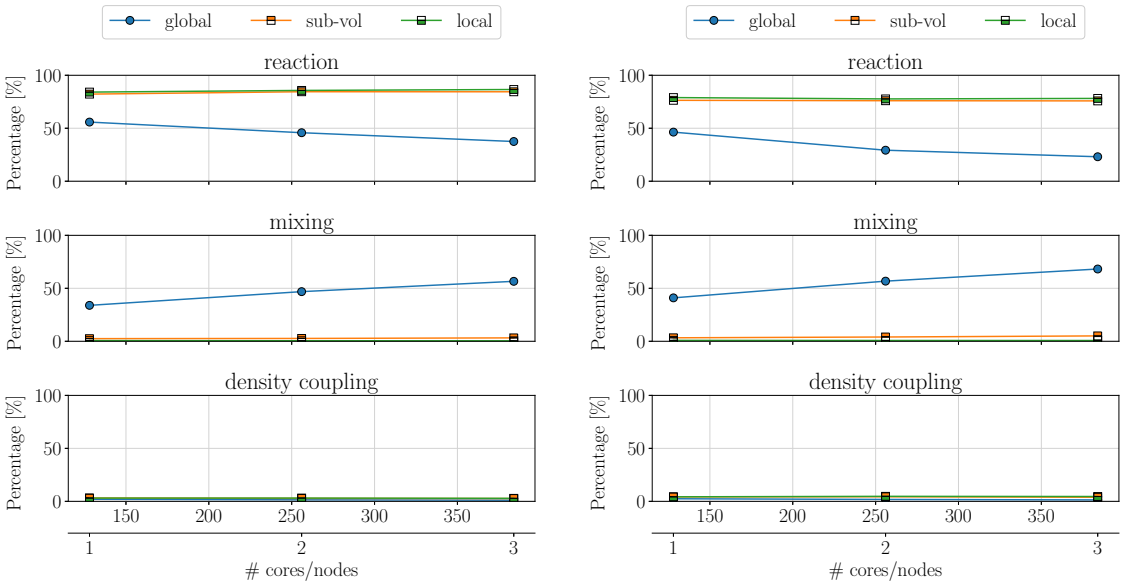
method is the newly implemented sub-volume option. The results emphasize the impact of the load balancing on the overall scaling as well as the choice of particle pairing. If the particle load balancing is switched off, solving the reaction scales poorly, and for the case of 358 cores, only 66% of the ideal scaling is reached. The reactive part's poor scaling coincides with the solver's overall scaling. The cause for this correlation becomes evident when the percentage of time spent in each of the three main components is investigated in Figure 5.12. The reactive part of the solver takes about 75% of the total time, except for the global pairing method. Hence a poor scaling of this part directly affects the total scaling of the solver. It further shows the significance of the particle pairing method, as the global pairing method does not scale and takes up 50% of the time for the case of 258 cores. Due to the inability to scale with more cores, the percentage of time spent on particle pairing and mixing increases with an increasing number of cores. The new sub-volume method also does not scale well, as the number of processors within one sub-volume is dependent on a physical size not on the domain decomposition. However, as only a small subset of all processors is considered for communication, the overall time spent on particle pairing and mixing is much less than for the global method, hence, that the impact on the total scaling is negligible. The local particle pairing method scales perfectly as the problem size decreases linearly with the number of cores. However, to reiterate, the local particle mixing is only valid for large processor domains, hence for a low decomposition, and is not a suitable approach for a HPC setup and decomposition into hundreds of cores. On the other hand, the sub-volume method considers multiple processors and scales as well as the local method. For the case of active load balancing, the results scale ideally up to the investigated decomposition of 384 cores.
In conclusion, the new mmcFoam solver combined with the sub-volume particle pairing and particle load balancing has a near perfect scaling within the investigated range and can be applied in an HPC setting.



(a) Without load balancing

(b) With load balancing

Figure 5.11: Total scaling of mmcFoam and scaling of main components, solving the reaction, particle pairing and mixing, and density coupling. Results are presented for the three different particle pairing methods without load balancing (a) and with load balancing (b). The black dashed line represents an ideal linear scaling.



(a) Without load balancing

(b) With load balancing

Figure 5.12: Percentage of the total time spent in each component of mmcFoam, solving the reaction, particle pairing and mixing, and density coupling. Results are presented for the three different particle pairing methods without load balancing (a) and with load balancing (b).

Bibliography

[1] M. J. Cleary and A. Y. Klimenko. "A Detailed Quantitative Analysis of Sparse-Lagrangian Filtered Density Function Simulations in Constant and Variable Density Reacting Jet Flows". In: *Physics of Fluids* 23.11 (Nov. 2011), p. 115102. ISSN: 1070-6631. DOI: 10.1063/1.3657085. URL: https://aip.scitation.org/doi/full/10.1063/1.3657085 (visited on 03/04/2023).

[2] M. J. Cleary and A. Y. Klimenko. "A Generalised Multiple Mapping Conditioning Approach for Turbulent Combustion". In: *Flow, Turbulence and Combustion* 82.4 (June 2009), pp. 477–491. ISSN: 1386-6184, 1573-1987. DOI: 10.1007/s10494-008-9161-3. URL: http://link.springer.com/10.1007/s10494-008-9161-3 (visited on 06/07/2023).

[3] R. L. Curl. "Dispersed Phase Mixing: I. Theory and Effects in Simple Reactors". In: *AIChE Journal* 9.2 (Mar. 1963), pp. 175–181. ISSN: 0001-1541, 1547-5905. DOI: 10.1002/aic.690090207. URL: https://onlinelibrary.wiley.com/doi/10.1002/aic.690090207 (visited on 06/07/2023).

[4] J. Janicka, W. Kolbe, and W. Kollmann. "Closure of the Transport Equation for the Probability Density Funcfion of Turbulent Scalar Fields". In: *Journal of Non-Equilibrium Thermodynamics* 4.1 (1979). ISSN: 0340-0204, 1437-4358. DOI: 10.1515/jnet.1979.4.1.47. URL: https://www.degruyter.com/document/doi/10.1515/jnet.1979.4.1.47/html (visited on 06/07/2023).

[5] J. Kirchmann et al. "Two-Phase Sparse-Lagrangian MMC-LES of Dilute Ethanol Spray Flames". In: *Proceedings of the Combustion Institute* 38.2 (Jan. 1, 2021), pp. 3343–3350. ISSN: 1540-7489. DOI: 10.1016/j.proci.2020.05.009. URL: https://www.sciencedirect.com/science/article/pii/S1540748920300092 (visited on 02/08/2023).

[6] H. W. Kuhn. "The Hungarian Method for the Assignment Problem". In: *Naval Research Logistics Quarterly* 2.1-2 (1955), pp. 83–97. ISSN: 1931-9193. DOI: 10.1002/nav.3800020109. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800020109 (visited on 05/30/2023).

[7] Gregor Neuber et al. "Joint Experimental and Numerical Study of Silica Particulate Synthesis in a Turbulent Reacting Jet". In: *Proceedings of the Combustion Institute* 37.1 (Jan. 1, 2019), pp. 1213–1220. ISSN: 1540-7489. DOI: 10.1016/j.proci.2018.06.074. URL: https://www.sciencedirect.com/science/article/pii/S1540748918302578 (visited on 02/03/2023).

[8] Gregor Neuber et al. "Sparse-Lagrangian MMC Modelling of the Sandia DME Flame Series". In: *Combustion and Flame* 208 (Oct. 2019), pp. 110–121. ISSN: 00102180. DOI: 10.1016/j.combustflame.2019.06.026. URL: https://linkinghub.elsevier.com/retrieve/pii/S0010218019302858 (visited on 10/09/2023).

[9] Son Vo et al. "Assessment of Mixing Time Scales for a Sparse Particle Method". In: *Combustion and Flame* 179 (May 1, 2017), pp. 280–299. ISSN: 0010-2180. DOI: 10.1016/j.combustflame.2017.02.017. URL: https://www.sciencedirect.com/science/article/pii/S0010218017300640 (visited on 02/08/2023).